

Adaptation Experiment 3: Learned Context Gates

1 Theoretical overview and goal

Retrieval can help some queries and harm others. A gate chooses between the vanilla forecast $\hat{y}_0$ and the context forecast $\hat{y}_c$. On T_2 the supervision signal is the loss advantage

$$\Delta_h = (y_h - \hat{y}_{0,h})^2 - (y_h - \hat{y}_{c,h})^2,$$

so $\Delta_h > 0$ means context is preferable. Scalar gates make one decision per window; horizon gates make one decision per forecast step. Regressors estimate Δ , while classifiers estimate the event $\Delta > 0$.

Experimental goal. Learn when retrieval should be trusted, quantify the value of query-level features, and compare attainable gates with target-aware oracle upper bounds.

2 Implementation details

The gate features summarize the context–vanilla difference, query mean and standard deviation, neighbor look-back statistics, same-user ratio, neighbor age, distance-weight concentration, and average distance. Horizon gates also receive the full horizon-wise context difference.

Gate	Training target	Decision granularity
Bayes no-feature	Mean T_2 advantage	Scalar / horizon
CatBoost classifier	$\mathbb{I}[\Delta > 0]$	Scalar / horizon
CatBoost regressor	Δ	Scalar / horizon
Oracle	True target loss	Scalar / horizon

CatBoost uses 300 iterations, learning rate 0.03, depth 4, balanced classes, and seed 1. Models are trained only on T_2 and applied without refitting to T_3 . An optional seeded maximum can subsample T_2 fitting rows from the saved extraction payload; it is unlimited by default, and final T_3 scoring always retains every extracted evaluation row. The small sweep uses three datasets, four settings, raw/instance Euclidean distance, $k \in \{1, 3, 10\}$, and Chronos. Full uses six datasets and adds 512–64 for Cross-RAG while remaining Chronos-only; ultra adds TabPFN-TS and skips complete Chronos outputs. TS-ICL remains a later extension.

Launcher: `gates.slurm`

Active module: `src.adaptors.baselines.evaluate -family gates`

Outputs include per-method metrics, prediction payloads, gate diagnostics, and feature importance plots.

The gate job loops sequentially and validates each extraction completion manifest before loading it. It must depend on the extraction job. The table job then checks every requested gate JSON before producing detailed and averaged tables for each backbone; missing configurations are errors rather than silent blanks.

3 Interpretation

The oracle gap measures headroom, not deployable performance. The comparison between the no-feature Bayes gate and feature-based gates isolates the benefit of conditional decisions. Final claims must use T_3 and should report both mean error and user-tail error because a gate can improve the mean while systematically rejecting useful context for a subgroup.